基于下一代硬件的Cube编程

作者 吴兆麟

时间 2026/03

课程背景与目标

课程背景:

- 随着昇腾AI处理器架构的持续演进，下一代芯片在硬件设计上对CUBE单元做了部分修改，包括算力提升、计算精度扩展、数据通路新增等;
- 这些硬件层面的变化直接影响了矩阵计算的编程及性能调优;

课程目标:

- 掌握昇腾下一代芯片的微架构升级要点，明确CUBE计算单元核心变化的技术逻辑;
- 具备基于下一代芯片特性的CUBE算子设计、开发与性能调优能力;

前置知识:

- 熟悉Ascend C基础语法、核函数开发流程，有当代CUBE算子(Matmul等）开发经验;
- 了解昇腾芯片基本架构(达芬奇架构、AICore单元）;

CANN

目录

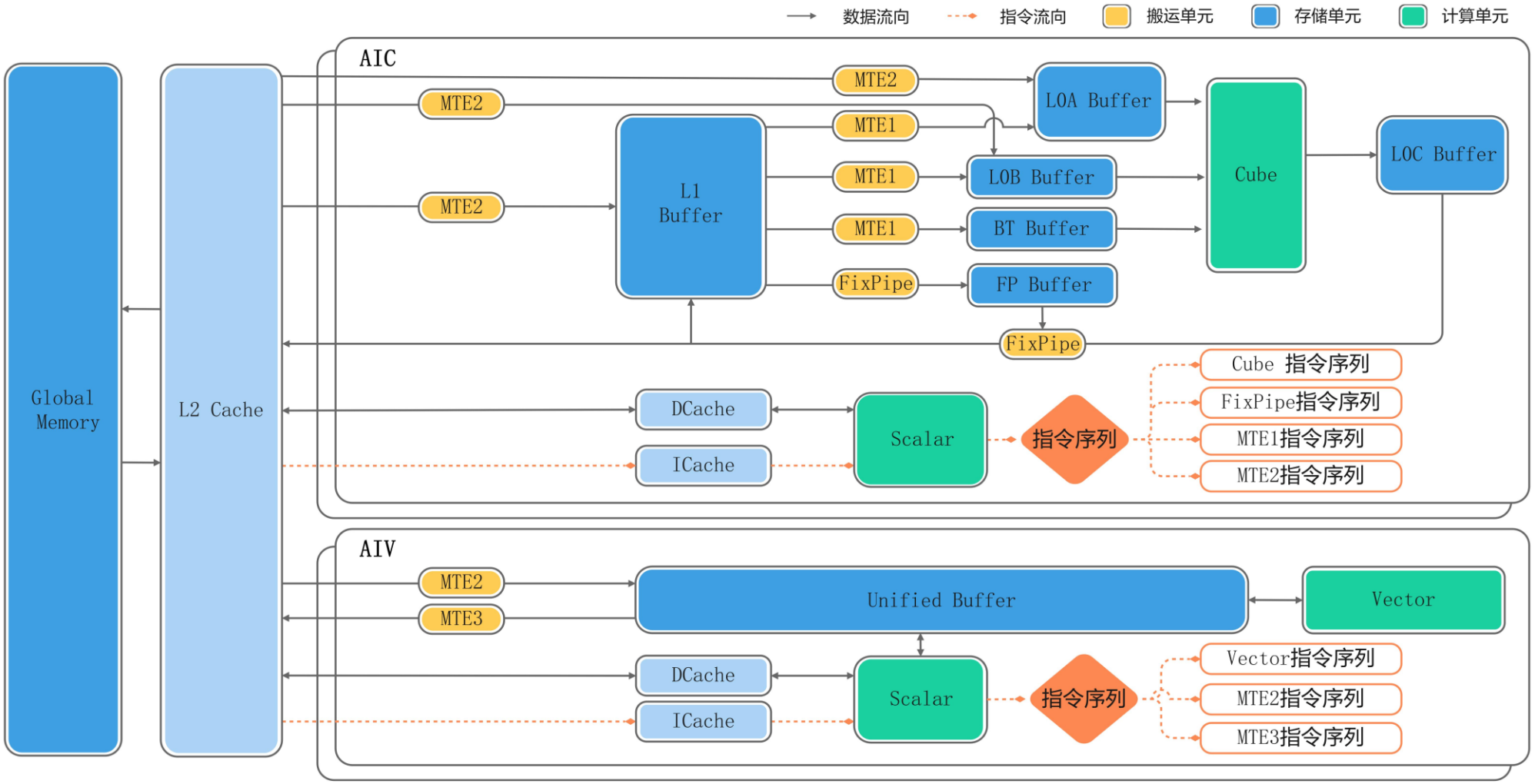
Part 1 当代CUBE编程基础回顾

Part 2 下一代CUBE单元升级解析

Part 3 下一代CUBE核心特性和编程API介绍

Part 4 下一代CUBE编程实操

当代CUBE编程基础回顾-芯片架构



CANN

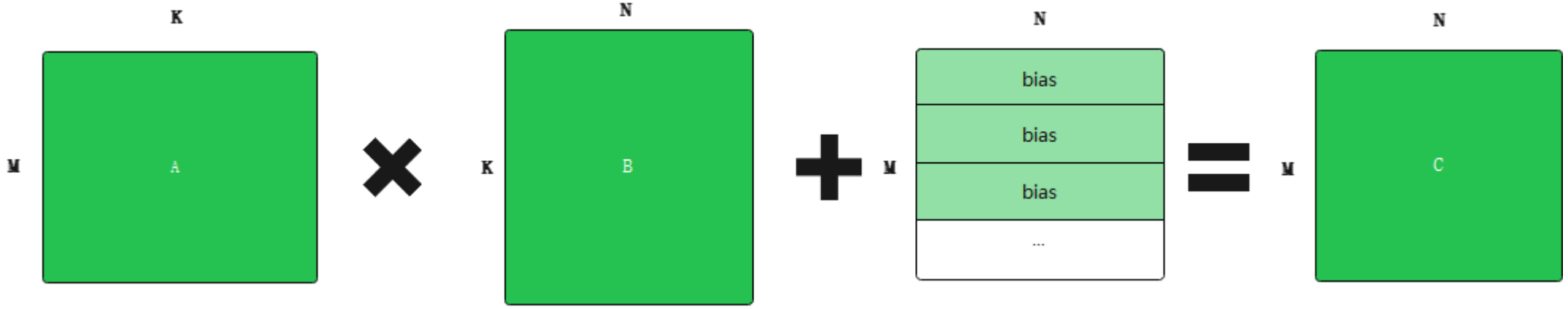
当代CUBE编程基础回顾-Matmul为例

Matmul的计算公式为：C = A * B + bias，其示意图如下。

•A、B为源操作数，A为左矩阵，形状为[M, K]；B为右矩阵，形状为[K, N]。

•C为目的操作数，存放矩阵乘结果的矩阵，形状为[M, N]。

• bias为矩阵乘偏置，形状为[1, N]。对A*B结果矩阵的每一行都采用该bias进行偏置。



CANN

当代CUBE编程基础回顾-Matmul为例(基础API)

```
// Kernel入口
__aicore__ inline void Init(GM_ADDR a, GM_ADDR b, GM_ADDR c)
{
    // ...
    aGM.SetGlobalBuffer((__gm__ half *)a);
    bGM.SetGlobalBuffer((__gm__ half *)b);
    cGM.SetGlobalBuffer((__gm__ float *)c);
    pipe.InitBuffer(inQueueA1, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueA2, 1, aSize * sizeof(half));
    pipe.InitBuffer(inQueueB1, 1, bSize * sizeof(half));
    pipe.InitBuffer(inQueueB2, 1, bSize * sizeof(half));
    pipe.InitBuffer(outQueueCO1, 1, cSize * sizeof(float));
}
__aicore__ inline void Process()
{
    CopyIn();
    SplitA();
    SplitB();
    Compute();
    CopyOut();
}
extern "C" __global__ __aicore__ void mmad_custom(GM_ADDR a,
GM_ADDR b, GM_ADDR c)
{
    KernelMmad op;
    op.Init(a, b, c);
    op.Process();
}
```

```
// CopyIn: 搬运GM(A, B矩阵)数据到L1(L1A & L1B)
__aicore__ inline void CopyIn()
{
    // ...
    AscendC::Nd2NzParams nd2nzA1Params;
    AscendC::DataCopy(a1Local, aGM, nd2nzA1Params);
    AscendC::Nd2NzParams nd2nzB1Params;
    AscendC::DataCopy(b1Local, bGM, nd2nzB1Params);
    // ...
}
```

```
// SplitA&SplitB: 将L1A&L1B的数据加载到L0A&L0B中
__aicore__ inline void SplitA()
{
    // ...
    AscendC::LoadData2DParamsV2 loadDataParams;
    AscendC::LoadData(a2Local, a1Local, loadDataParams);
    // ...
}
```

```
// Compute: 进行Mmad计算, 将L0A&L0B中数据进行Mmad计算
__aicore__ inline void Compute()
{
    // ...
    AscendC::MmadParams mmadParams;
    mmadParams.m = m;
    mmadParams.n = n;
    mmadParams.k = k;
    AscendC::Mmad(c1Local, a2Local, b2Local, mmadParams);
    // ...
}
```

```
// CopyOut: 将矩阵计算结果从L0C中搬出到GM中
__aicore__ inline void CopyOut()
{
    // ...
    AscendC::FixpipeParamsV220 fixpipeParams;
    // 配置Fixpipe的参数
    AscendC::Fixpipe(cGM, c1Local, fixpipeParams);
    // ...
}
```

示例来源： asc-devkit-dev/examples/03_libraries/01_matrix/mmad · CANN/asc-devkit-dev - AtomGit | GitCode

当代CUBE编程基础回顾-Matmul为例(高阶API)

```
__global__ __aicore__ void matmul_custom(GM_ADDR a, GM_ADDR b, GM_ADDR c, __kfc_workspace__ GM_ADDR workspace,GM_ADDR tilingGm)
{
    // ...
    aGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ A_T *>(a), tiling.M * tiling.Ka);
    bGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ B_T *>(b), tiling.Ka * tiling.N);
    cGlobal.SetGlobalBuffer(reinterpret_cast<__gm__ C_T *>(c), tiling.M * tiling.N);
    // ...
    // Calculate the gm offset and tail size based on the blockidx.
    CalcGMOffset(GetBlockIdx(), tiling, offsetA, offsetB, offsetC, tailM, tailN, isTransA, isTransB);

    auto gmA = aGlobal[offsetA];
    auto gmB = bGlobal[offsetB];
    auto gmC = cGlobal[offsetC];

    Matmul<MatmulType<AscendC::TPosition::GM, CubeFormat::ND, A_T>,
        MatmulType<AscendC::TPosition::GM, CubeFormat::ND, B_T>,
        MatmulType<AscendC::TPosition::GM, CubeFormat::ND, C_T>> mm;
    REGIST_MATMUL_OBJ(&pipe, GetSysWorkSpacePtr(), mm, &tiling); // Initialize the matmul object.
    if (GetBlockIdx() >= tiling.usedCoreNum) {
        return;
    }
    mm.SetOrgShape(tiling.M, tiling.N, tiling.Ka, tiling.Kb);
    mm.SetTensorA(gmA, isTransA);
    mm.SetTensorB(gmB, isTransB);
    mm.SetTail(tailM, tailN);
    mm.IterateAll(gmC);
    mm.End();
}
```

示例来源：asc-devkit/examples · CANN/asc-devkit - AtomGit | GitCode

CANN

本章小结

- **当代CUBE编程的知识点**
  - 当代CUBE侧的微架构, 分离架构，无CV直连通道;
  - CUBE编程的编程接口：基础API接口和高阶API接口;

CANN

目录

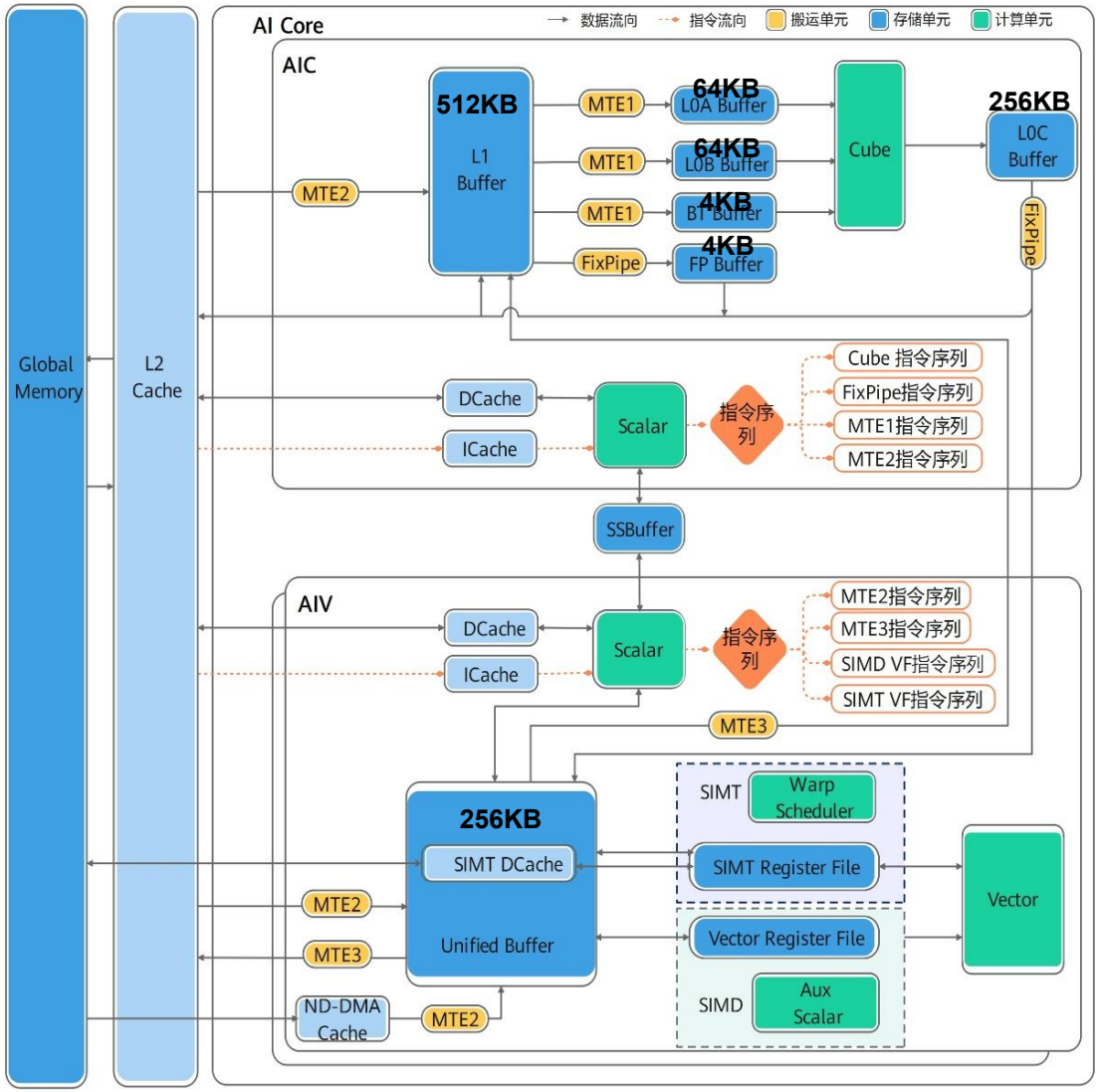
Part 1 当代CUBE编程基础回顾

Part 2 下一代CUBE单元升级解析

Part 3 下一代CUBE核心特性和编程API介绍

Part 4 下一代CUBE编程实操

下一代CUBE单元升级解析-新一代微架构解读



影响CUBE编程的芯片变更:

1. 更强的存储能力:

1.1 调整: 核的总数增加(24->32);

1.2 调整: L0C、FB、BT、UB存储空间增大。

2. 数据搬运通路变更:

2.1 新增: L0C Buffer -> Unified Buffer、Unified Buffer <-> L1 Buffer的数据通路。

2.2 删除: Global Memory -> L0A Buffer、Global Memory -> L0B Buffer的数据通路。

2.3 删除: L1 Buffer-> Global Memory的数据通路。

3. 新的搬运指令和能力:

3.1 新增: DN2NZ以及Fixpipe随路的NZ2DN指令支持NCHW格式。

3.2 新增: Load指令新增V2模式，增加repeat-stride和for-loop能力。

3.3 新增: 增加双目标输出模式, L0C->AIV0.UB+AIV1.UB, 支持一条指令同时完成2路拷贝。

4. 新支持的数据类型:

4.1 新增: 支持HiF8和MXFP等8bit和4bit数据类型。

5. 硬件数据排布格式变化:

5.1 调整: L0A的分型从ZZ变成NZ。

6. 新的专用存储单元:

6.1 新增: SSBuffer，用于AIC和AIV的核间通信。

7. 新的核间同步指令:

7.1 新增: 硬件新增CV核间同步指令, 支持AIV0/AIV1单独触发AIC的等待。

CANN

本章小结

- **下一代芯片对CUBE单元的升级**
  - 内存大小的变化，L0C&FB&BT&UB扩容；
  - 新增UB2L1和L0C2UB的通路；
  - 新增低比特数据类型(FP4, FP8)；
  - 新增DN2NZ指令、L0A分型从Zz变更为Nz；

CANN

目录

Part 1 当代CUBE编程基础回顾

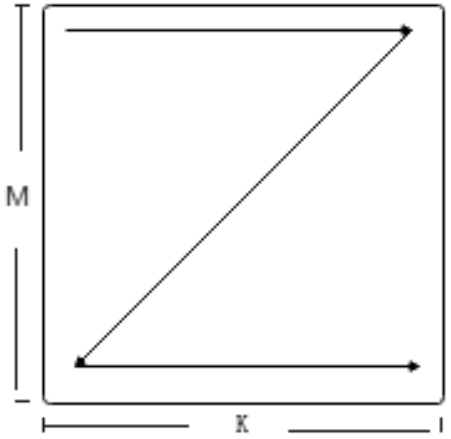
Part 2 下一代CUBE单元升级解析

Part 3 下一代CUBE核心特性和编程API介绍

Part 4 下一代CUBE编程实操

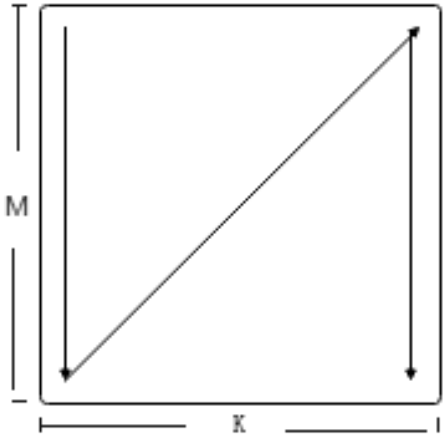
下一代CUBE编程核心特性-新增DN分型

新增DN数据格式，并且数据搬入和数据搬出新增支持DN2NZ和NZ2DN的接口能力，对于卷积类算子部分场景无需插入TransData，可以直接通过搬运接口进行随路的转置。

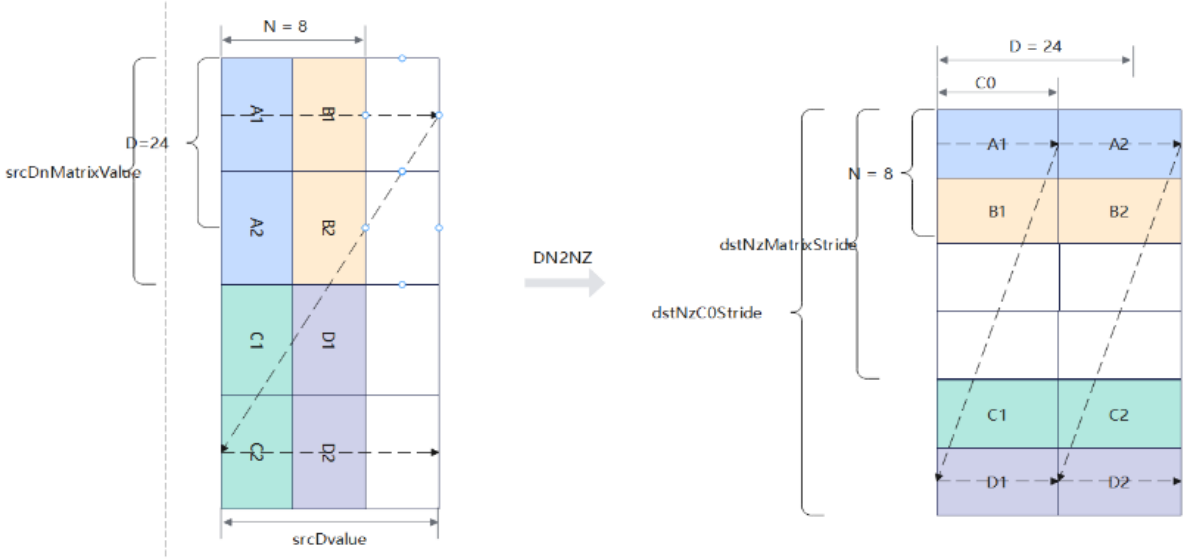


ND格式

```
// 1. 新增: GM2L1的Dn2Nz搬运接口
template <typename T, bool enableSmallC0 = false>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const GlobalTensor<T>& src, const Dn2NzParams& intriParams);
struct Dn2NzParams {
  uint16_t dnNum = 0;
  uint16_t nValue = 0;
  uint32_t dValue = 0;
  uint64_t srcDnMatrixStride = 0;
  uint64_t srcDValue = 0;
  uint16_t dstNzC0Stride = 0;
  uint16_t dstNzNStride = 0;
  uint32_t dstNzMatrixStride = 0;
};
```



DN格式



CANN

下一代CUBE编程核心特性-低比特数据类型支持(1)

矩阵乘新增FP4、FP8数据类型，通过可控的精度损失，实现算力吞吐倍增、内存占用减半及能效显著提升，为大模型训练与推理提供关键的低功耗高性能计算基础。

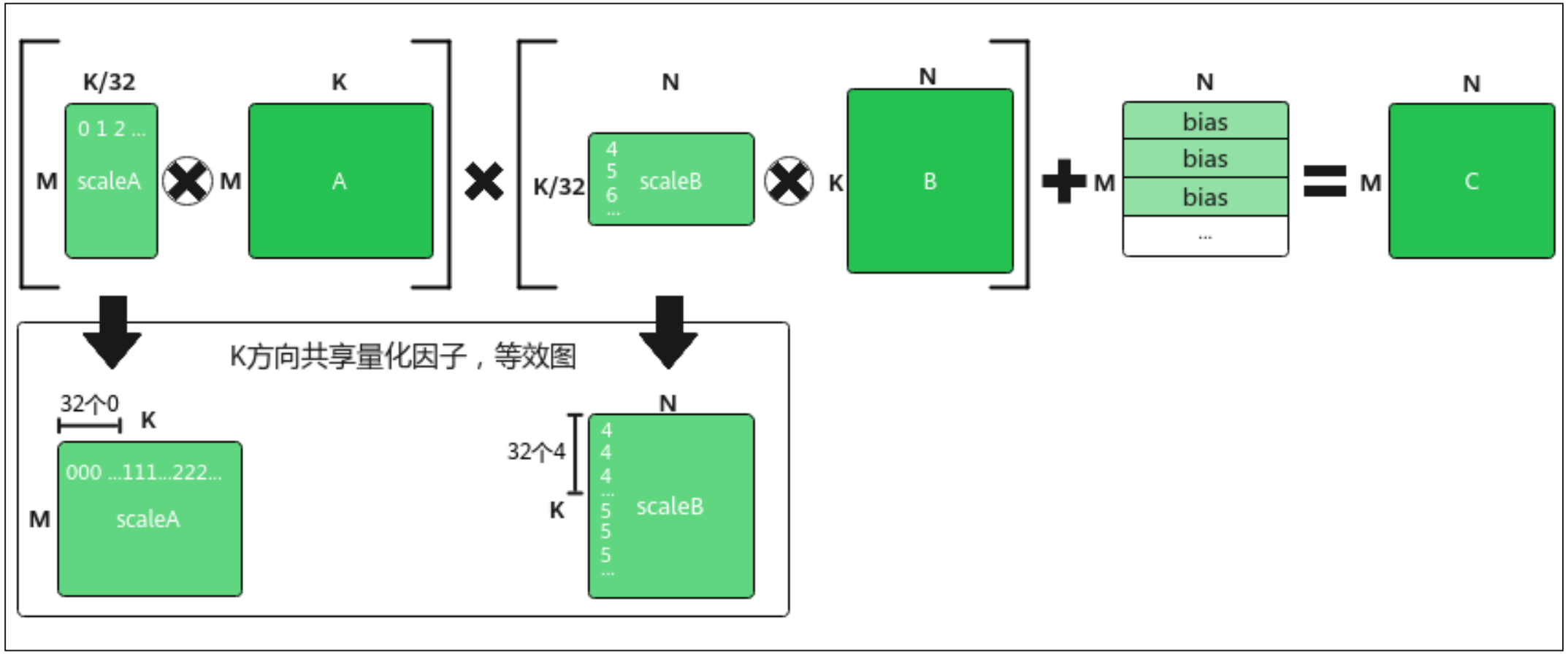
不含Scale缩放的矩阵乘支持的数据类型			
左矩阵	右矩阵	Bias	结果矩阵
fp8_e4m3fn_t	fp8_e4m3fn_t	float	float
fp8_e4m3fn_t	fp8_e5m2_t	float	float
fp8_e5m2_t	fp8_e4m3fn_t	float	float
fp8_e5m2_t	fp8_e5m2_t	float	float
hifloat8_t	hifloat8_t	float	float

包含Scale缩放的矩阵乘支持的数据类型					
左矩阵	左矩阵Scale	右矩阵	右矩阵Scale	Bias	结果矩阵
fp4x2_e1m2_t	fp8_e8m0_t	fp4x2_e1m2_t	fp8_e8m0_t	float	float
fp4x2_e1m2_t	fp8_e8m0_t	fp4x2_e1m2_t	fp8_e8m0_t	float	float
fp4x2_e2m1_t	fp8_e8m0_t	fp4x2_e1m2_t	fp8_e8m0_t	float	float
fp4x2_e1m2_t	fp8_e8m0_t	fp4x2_e2m1_t	fp8_e8m0_t	float	float
fp4x2_e2m1_t	fp8_e8m0_t	fp4x2_e2m1_t	fp8_e8m0_t	float	float
mx_fp8_e4m3_t	fp8_e8m0_t	mx_fp8_e4m3_t	fp8_e8m0_t	float	float
mx_fp8_e4m3_t	fp8_e8m0_t	mx_fp8_e5m2_t	fp8_e8m0_t	float	float
mx_fp8_e5m2_t	fp8_e8m0_t	mx_fp8_e4m3_t	fp8_e8m0_t	float	float
mx_fp8_e5m2_t	fp8_e8m0_t	mx_fp8_e5m2_t	fp8_e8m0_t	float	float

CANN

下一代CUBE编程核心特性-低比特数据类型(2)

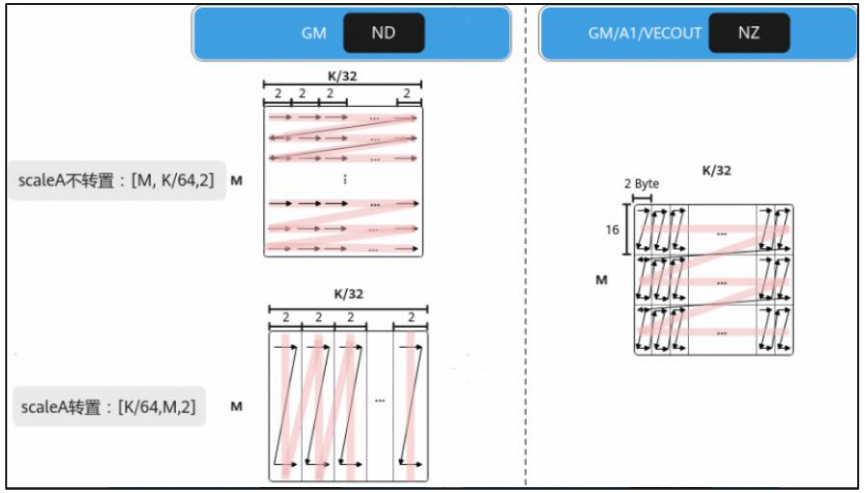
包含缩放功能的矩阵乘，公式如下：C = (ScaleA ⊗ A) ∗ (ScaleB ⊗ B) + C。ScaleA和ScaleB通过LoadData2DMX接口载入。



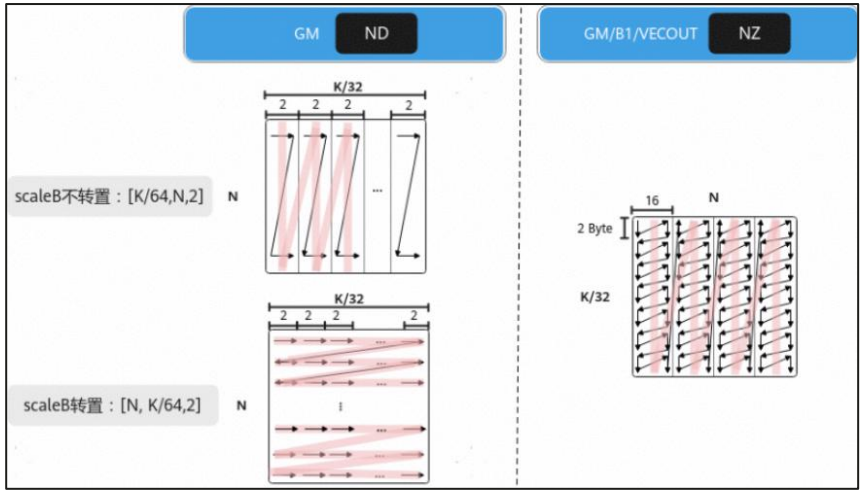
MxMatmul矩阵乘示意图

CANN

下一代CUBE编程核心特性-低比特数据类型(3)



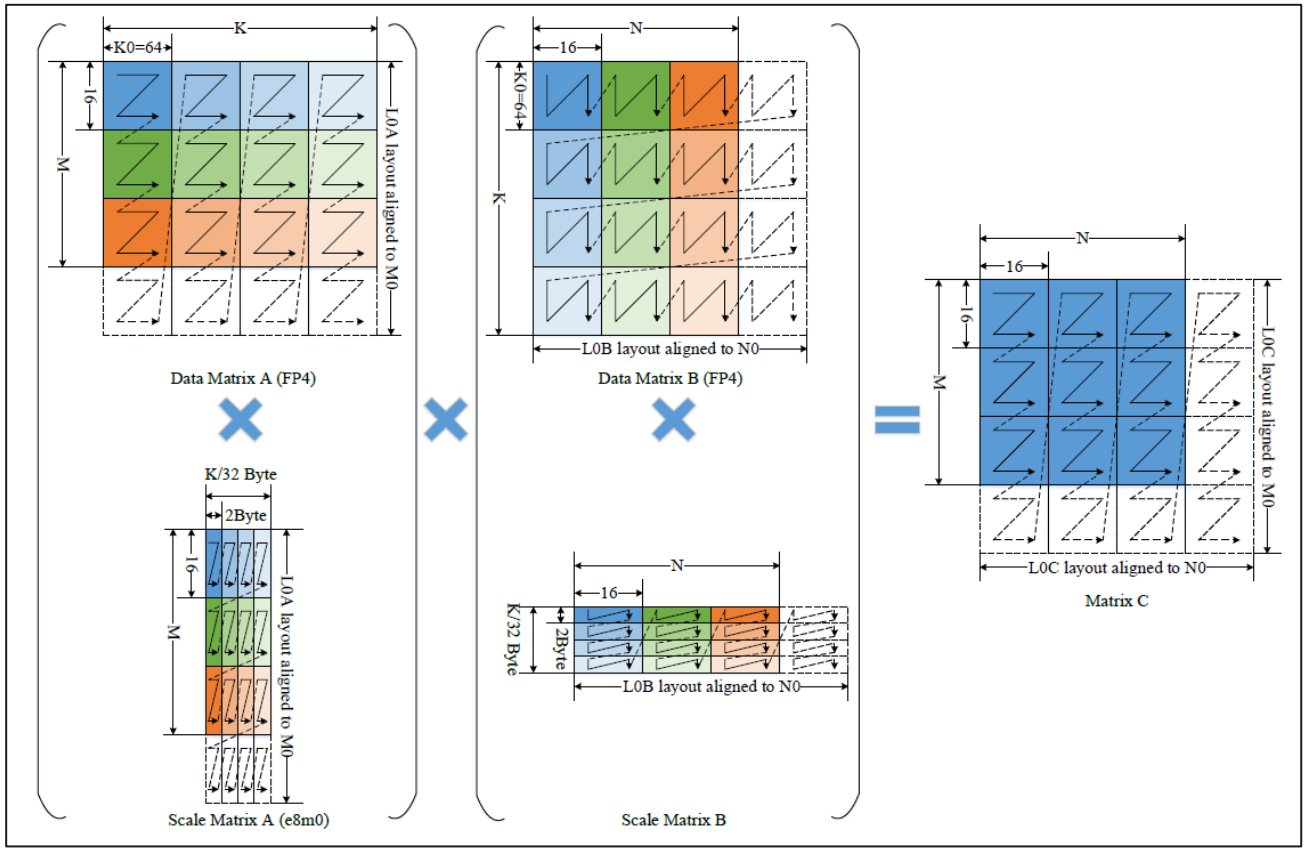
ScaleA 在GM&L1中的排布



ScaleB 在GM&L1中的排布

Scale的数据相对于A, B矩阵的排布有一些特定需求, 主要如下图所示:

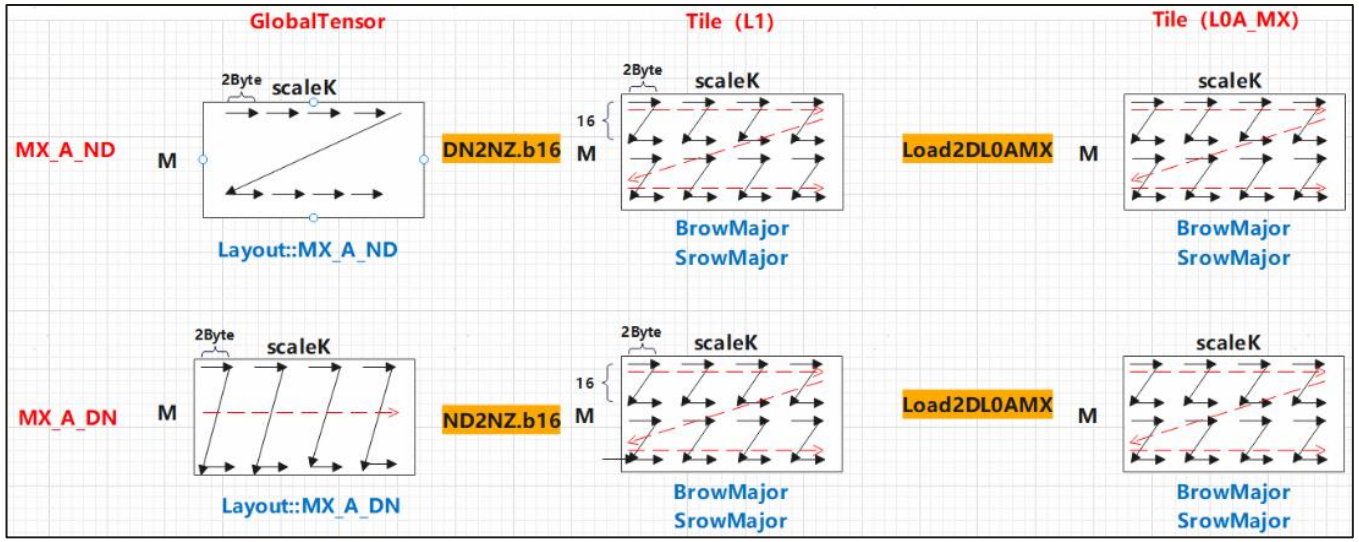
- ScaleA的分形格式为小Z大Z， shape为（16，2），数据类型为fp8_e8m0_t。
- ScaleB的分形格式为小N大N，shape为（2，16），数据类型为fp8_e8m0_t。

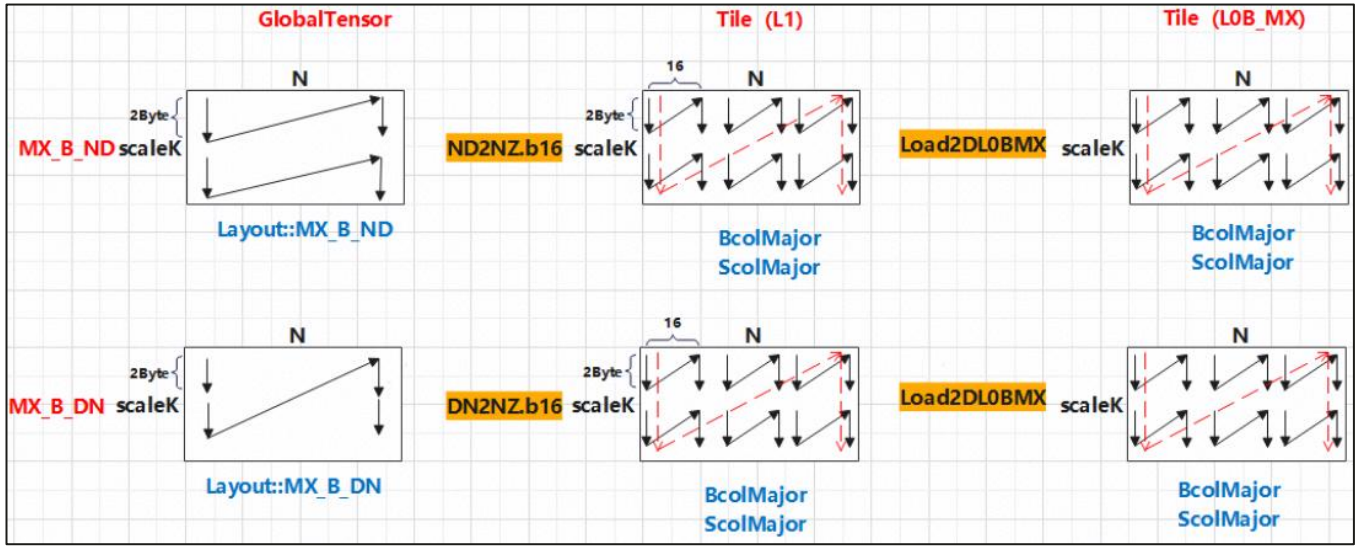


带Scale缩放能力的矩阵乘法示例(FP4为例)

CANN

下一代CUBE编程核心特性-低比特数据类型(4)





结合GM2L1的随路ND2NZ & DN2NZ的能力，ScaleA & ScaleB在转置/非转置场景下可以分别通过随路搬运能力进行处理，具体细分搬运能力如左图所示：

- **ScaleA非转置**: GMScaleA[M, K / 64, 2] 通过DataCopy(DN2NZ.b16)能力进行搬运到L1，并通过LoadData加在到L0A[M1, K/64, M0, 2]中；
- **ScaleA转置**: GMScaleA[K / 64, M, 2] 通过DataCopy(ND2NZ.b16)能力进行搬运到L1，并通过LoadData加在到L0A[M1, K/64, M0, 2]中；

- **ScaleB非转置**: GMScaleB[K / 64, N, 2] 通过DataCopy(ND2NZ.b16)能力进行搬运到L1，并通过LoadData加在到L0B[N1, K/64, N0, 2]中；
- **ScaleB转置**: GMScaleB[N, K / 64, 2] 通过DataCopy(DN2NZ.b16)能力进行搬运到L1，并通过LoadData加在到L0B[N1, K/64, N0, 2]中；

CANN

下一代CUBE编程核心特性-低比特数据类型(5)

能力使能关键接口能力

1. 关键接口

1.1 DataCopy

ScaleA & ScaleB使用Nd2Nz/Dn2Nz的接口能力搬运到L1上，接口无需特殊处理；

1.2 LoadData

L1上的Buffer内容会加载到特定的Buffer上进行处理，用户当前不感知，通过LoadData接口使能

```
template <typename T, typename U = T>
__aicore__ inline void LoadData(const LocalTensor<U>& dst, const LocalTensor<T>& src,
const LocalTensor<fp8_e8m0_t>& srcMx,
const LoadData2DParamsV2& loadDataParams,
const LoadData2DMxParams& loadMxDataParams)
```

新增:

srcMx: L1上面的Scale的内存

loadMxDataParams: MX数据相关的配置(xPosition,yPositon等)

1.3 Mmad

Mmad接口根据MXFP的数据类型自动执行对应的Mmad指令行为，接口无特定变更；

1.4 FixPipe

Scale的能力不涉及FixPipe搬出，因此无特定变更；

核心代码片段

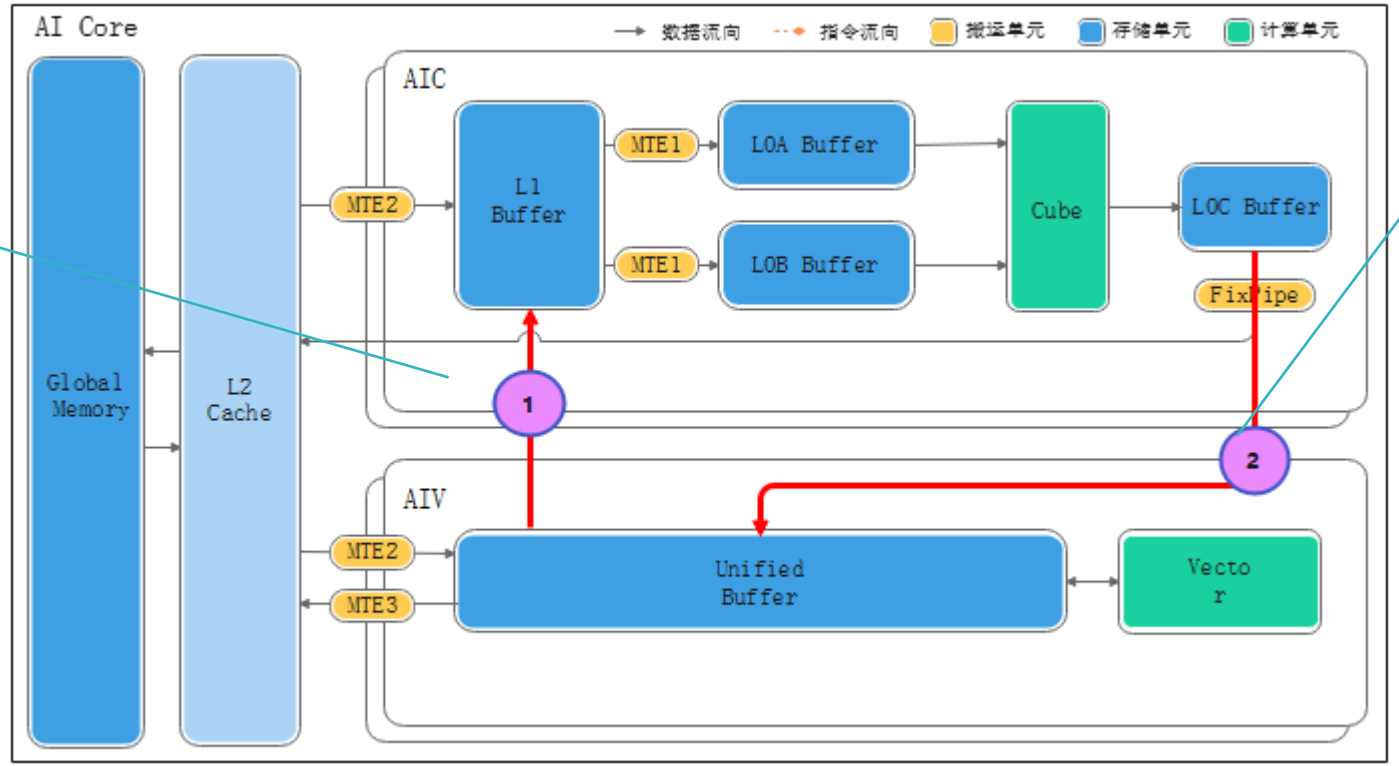
```
__aicore__ inline void MatmulKernel(const GlobalTensor<DstT> &gm, bool transpose) {
  // ...
  // 1. 搬运阶段: 除了搬运A, B 矩阵外，需要搬运ScaleA & ScaleB矩阵，搬运规则如上一页所示，本例中ScaleA&ScaleB都为非转置
  // ScaleA为非转置, 使用DN2NZ的能力搬运
  LocalTensor<fp8_e8m0_t> l1AScale = qidA1MX_.template AllocTensor<fp8_e8m0_t>();
  auto workLocal = l1AScale.ReinterpretCast<half>();
  Dn2NzParams dn2nzParams = { /*...*/ };
  DataCopy(workLocal, gmAScale, dn2nzParams);
  // ScaleB为非转置, 使用ND2NZ的能力搬运
  LocalTensor<fp8_e8m0_t> l1BScale = qidB1MX_.template AllocTensor<fp8_e8m0_t>();
  Nd2NzParams nd2nzParams = { /*...*/ };
  DataCopy(l1BScale, gmBScale, kLength * nLength / 32);
  // 2. 加载阶段: 通过扩展的LoadData接口，对A & AScale, B & BScale加在到L0A&L0B中, 其中Scale的数据加载到特殊寄存器上
  LoadData2DParamsV2 loadDataParams = { /*...*/ };
  LoadData2DMxParams loadDataMxParams = { /*...*/ };
  LoadData(l0A, l1A, l1AScale, loadDataParams, loadDataMxParams);
  // ...
  LoadData2DParamsV2 loadDataParams = { /*...*/ };
  LoadData2DMxParams loadDataMxParams = { /*...*/ };
  LoadData(l0B, l1B, l1BScale, loadDataParams, loadDataMxParams);

  // 3. 计算阶段: MX的相关计算场景跟普通场景并无太大差别
  MmadParams mmadParams = { /*...*/ };
  Mmad(l0C, l0A, l0B, mmadParams);

  // 4. 搬出阶段：MX场景对于搬出阶段也无任何特殊点
  FixpipeParamsC310<CO2Layout::ROW_MAJOR> fixpipeParams(nLength, mLength, mLength, nLength);
  fixpipeParams.params = { 1, 0, 0 };
  Fixpipe<DstT, L0cT, CFG_ROW_MAJOR>(gmC, l0C, fixpipeParams);
}
```

CANN

下一代CUBE编程核心特性-更强的CV通路能力(1)

新架构引入UB2L1&L12UB&L0C2UB间的直连通路，实现矩阵计算数据的快速搬移，旨在简化CV融合算子开发并提升性能。

矩阵搬入: 启用UB至L1(UB2L1）直连通路，通过DataCopy接口，支持融合算子的向量计算结果直接搬入L1。



矩阵搬出: 启用L0C至UB(L0C2UB)直连通路，通过DataCopy接口，支持融合算子的矩阵计算结果直接搬入UB进行后续向量计算。

AI Core内 新增CV交互的通路

CANN

下一代CUBE编程核心特性-更强的CV通路能力(2)

关键使能接口定义

1. 新增: 搬入接口增加UB2L1的Nd2Nz搬入，支持Src&Dst都是LocalTensor的形式

```
template <typename T>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const LocalTensor<T>& src, const Nd2NzParams& intriParams);
```

2. 新增: 搬出接口增加L0C2UB的搬出, 支持直接从L0C搬出到UB, 支持Src&Dst都是LocalTensor的形式

```
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const LocalTensor<T>& dst, const LocalTensor<U>& src, const FixpipeParamsC310<config.format>& intriParams)
```

3. 能力增强: 核间同步接口新增模式4

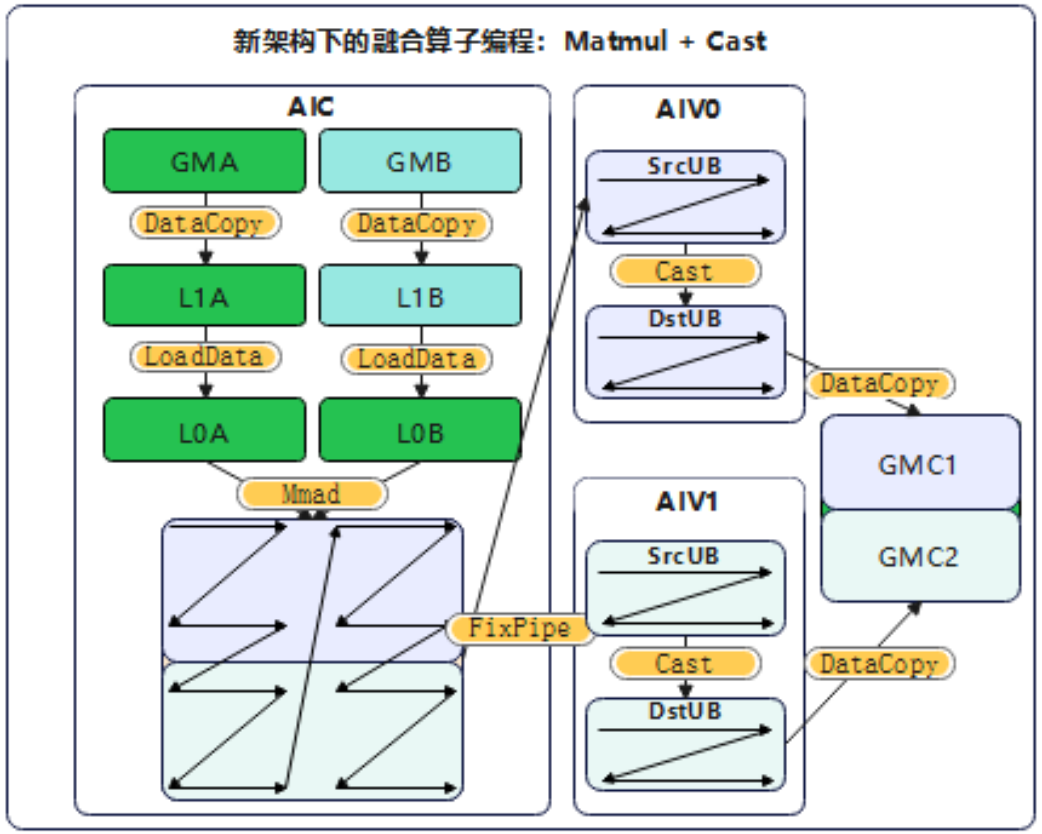
```
template <uint8_t modeId, pipe_t pipe>
__aicore__ inline void CrossCoreSetFlag(uint16_t flagId)

template <uint8_t modeId = 0, pipe_t pipe = PIPE_S>
__aicore__ inline void CrossCoreWaitFlag(uint16_t flagId)
```

modeId增强: 模式4：AI Core内部，AIC与AIV之间的同步控制。AIV0与AIV1可单独触发AIC等待。
CrossCoreWaitFlag新增等待流水, 新架构支持配置除PIPE_S外的其他流水，能够更好的进行流水间的并行。

CANN

下一代CUBE编程核心特性-更强的CV通路能力(3)

新架构新增的优化特性显著提升了CV流水并行效率，通过针对性优化流水同步机制，有效降低了同步开销，更有利于融合算子的性能优化。



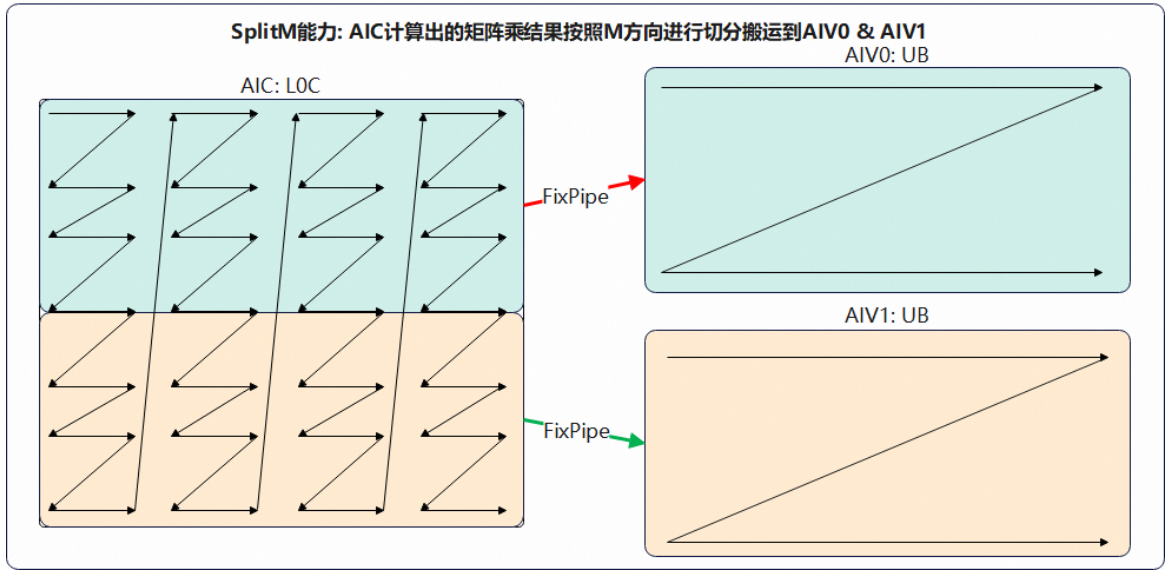

```
// AIC代码示例(关注计算流和核间同步，忽略核内同步)
__aicore__ inline void CubeOperator(const LocalTensor<DstT> &ubBuffer, bool transpose) {
    // ...
    // 1. 搬运和加载阶段忽略
    // ...
    // 3. 计算阶段: 使用MMad进行计算
    MmadParams mmadParams = { /*...*/ };
    Mmad(l0C, l0A, l0B, mmadParams);
    // 4. 搬出阶段：将计算结果搬出到UB中
    FixpipeParamsC310<CO2Layout::ROW_MAJOR> fixpipeParams(nLength, mLength, mLength,
nLength);
    fixpipeParams.params = { 1, 0, 0 };
    // 调用Fixpipe直接输出到UB中
    Fixpipe<DstT, L0cT, CFG_ROW_MAJOR>(ubBuffer, l0C, fixpipeParams);
    // 5. 通知Vector
    CrossCoreSetFlag<SYNC_MODE4, PIPE_FIX>(SYNC_AIV_AIC_FLAG);
}
```

```
// AIV代码示例(关注计算流和核间同步，忽略核内同步)
__aicore__ inline void CubeOperator(const LocalTensor<DstT> &ubBuffer, bool transpose) {
    // ...
    // 1. 等待Cube通知
    CrossCoreWaitFlag<SYNC_MODE4, PIPE_V>(SYNC_AIV_AIC_FLAG);
    // ...
    // 3. 使用UBBuffer进行计算处理
    // Vector计算
}
```

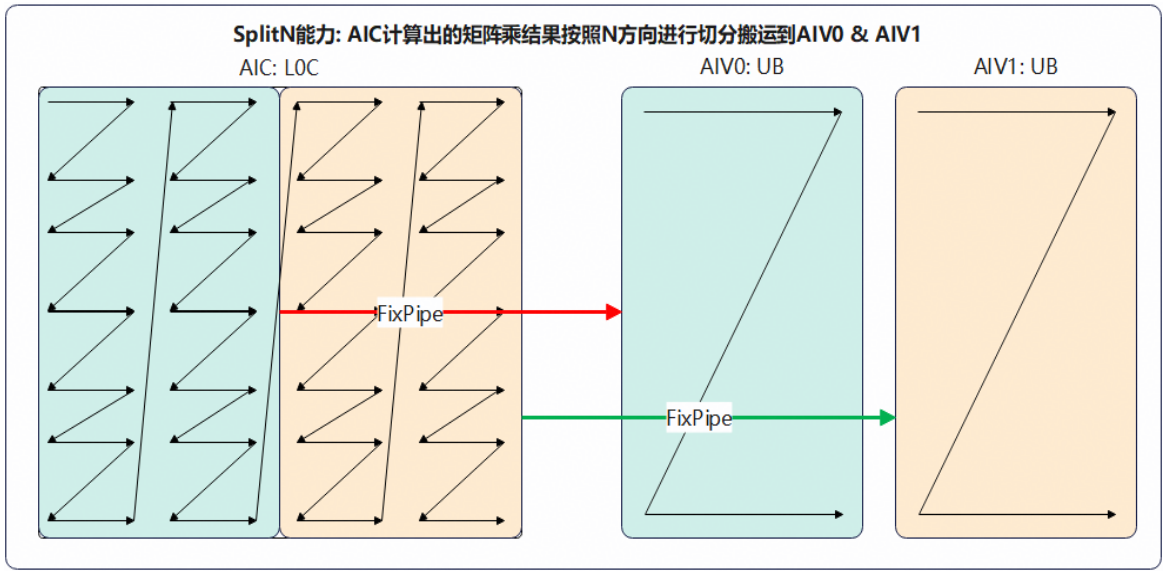
CANN

下一代CUBE编程核心特性-DualDst模式(1)

双目标模式控制, 将L0C中的M×N矩阵将被分成两半, 并同时写入两个AIV核的UB中。



- 按M维度拆分， M / 2 * N写入每个UB, M必须为2的倍数。



- 按N维度拆分， M * N/2写入每个UB, N必须为32的倍数。

CANN

下一代CUBE编程核心特性-DualDst模式(2)

```
// 使能接口定义
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const LocalTensor<T>& dst, const LocalTensor<U>& src, const FixpipeParamsC310<config.format>& intriParams)
// Fixpipe的参数说明
template <CO2Layout format = CO2Layout::ROW_MAJOR>
struct FixpipeParamsC310 {
  // ...
  uint8_t dualDstCtl = 0;
  // ...
};
```

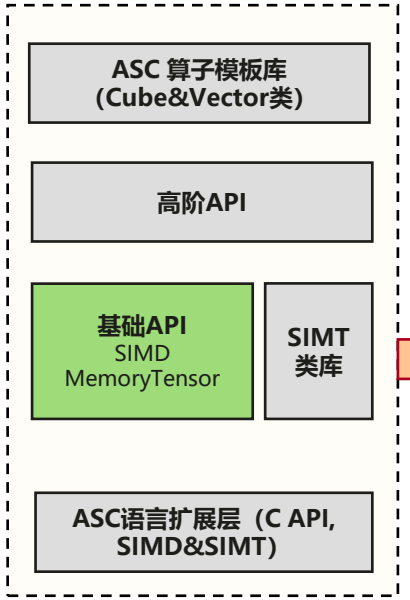
- dualDstCtl = 2'b00, 单目标模式，将整个矩阵写入通过subBlockId参数配置的目标UB。
- dualDstCtl = 2'b01：双目标模式，按M维度拆分，M / 2 * N写入每个UB, M必须为2的倍数。
- dualDstCtl = 2'b10：双目标模式，按N维度拆分，M * N / 2写入每个UB, N须为32的倍数。
- dualDstCtl = 2'b11：保留。

```
// 双目标模式使能代码示例
template <typename UBType, typename L0CType>
__aicore__ inline void CopyL0CToUB(const LocalTensor<UBType> &ubLocal, const LocalTensor<L0CType> &l0cLocal) {
  // ... nSizeIn, mSizeIn, srcStrideIn, dstStrideIn等参数计算(略)
  AscendC::FixpipeParamsC310<AscendC::CO2Layout::ROW_MAJOR> fixpipeParams(nSizeIn, mSizeIn, srcStrideIn, dstStrideIn);
  // 设置dualDstCtl能力, 当前设置值为0b01, 表示按照M维度进行切分
  fixpipeParams.dualDstCtl = 0b01;
  // 调用Fixpipe接口, 实现将L0C的结果写入到AIV0 & AIV1的UB地址中(AIV0 & AIV1的UB地址一致)
  AscendC::Fixpipe<UBType, L0CType, CFG_ROW_MAJOR_UB>(ubLocal, l0cLocal, fixpipeParams);
}
```

CANN

下一代CUBE编程语法及主要接口-基础API(1)

基础API层封装芯片底层能力，映射架构变更，助力开发者直接调用硬件资源，释放芯片极致性能。



1. 搬入接口：新增UB2L1接口，新增Gm2L1的DN2NZ

```
// 1. 保持: GM2L1的Nd2Nz搬运接口
template <typename T, bool enableSmallC0 = false>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const GlobalTensor<T>& src, const Nd2NzParams& intriParams)

// 2. 新增: UB2L1的Nd2Nz搬运接口
template <typename T>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const LocalTensor<T>& src, const Nd2NzParams& intriParams)

// 3. 新增: GM2L1的Dn2Nz搬运接口
template <typename T, bool enableSmallC0 = false>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const GlobalTensor<T>& src, const Dn2NzParams& intriParams);
```

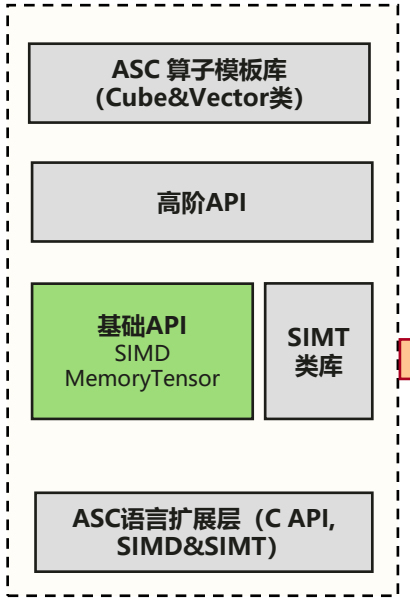
2. 加载接口：新增ParamsV2能力，新增MX的加载接口，去除GM2L0A/B的加载接口

```
// 1. L12L0A/B的加载接口, 跟A3保持兼容
template <typename T>
__aicore__ inline void LoadData(const LocalTensor<T>& dst, const LocalTensor<T>& src, const LoadData2DParams& loadDataParams)

// 2.新增: L12L0A/B的能力增强加载接口,
template <typename T>
__aicore__ inline void LoadData(const LocalTensor<T>& dst, const LocalTensor<T>& src, const LoadData2DParamsV2& loadDataParams)

// 3.新增: 新增左右矩阵MX随路加载接口
template <typename T, typename U = T>
__aicore__ inline void LoadData(const LocalTensor<U>& dst, const LocalTensor<T>& src, const LocalTensor<fp8_e8m0_t>& srcMx, const
LoadData2DParamsV2& loadDataParams, const LoadData2DMxParams& loadMxDataParams)
```

CANN

下一代CUBE编程语法及主要接口-基础API(2)

基础API层封装芯片底层能力，映射架构变更，助力开发者直接调用硬件资源，释放芯片极致性能。



3. 计算接口：基础Mmad计算保持跟A3兼容，去除Sparse相关能力

```
// 1. 保持: 基础Mmad计算接口
template <typename T, typename U, typename S>
__aicore__ inline void Mmad(const LocalTensor<T>& dst, const LocalTensor<U>& fm, const LocalTensor<S>& filter, const MmadParams&
mmadParams)

// 2. 保持: 包含Bias的Mmad计算接口
template <typename T, typename U, typename S, typename V>
__aicore__ inline void Mmad(const LocalTensor<T>& dst, const LocalTensor<U>& fm, const LocalTensor<S>& filter, const LocalTensor<V>&
bias, const MmadParams& mmadParams)
```

4. 搬出接口：新增L0C2UB的能力，扩展FixpipeParams结构体，补充双目标搬出等能力

```
// 1. 新增: L0C2GM搬出接口, 扩展FixpipeParams
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const GlobalTensor<T>& dst,
const LocalTensor<U>& src,
const FixpipeParamsC310<config.format>& intriParams)

// 2. 新增: L0C2UB搬出接口, 支持直接从L0C搬出到UB
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const LocalTensor<T>& dst,
const LocalTensor<U>& src,
const FixpipeParamsC310<config.format>& intriParams)
```

```
// 扩展Fixpipe结构体，新增双目标搬出能力及
Nz2Dn能力
template <CO2Layout format =
CO2Layout::ROW_MAJOR>
struct FixpipeParamsC310 {
    // ...
    uint8_t dualDstCtl = 0;
    bool subBlockId = false;
    typename
TransformParams<format>::PARAMS
params;
};
```

CANN

下一代CUBE编程语法及主要接口-Tensor API

基础API 扩展支持Tensor API，Tensor支持Layout，简化基于Tensor的计算接口，匹配业界Tensor编程体验

- ASC 算子模板库 (Cube&Vector类)
- 高阶API
- 基础API SIMD MemoryTensor
- SIMT API
- ASC语言扩展层（C API, SIMD&SIMT）

Shape&Stride

```
template <typename... Shapes>
using Shape = Std::tuple<Shapes...>;
template <typename... Strides>
using Stride = Std::tuple<Strides...>;
```

Layout

```
template <typename ShapeType, typename StrideType>
struct Layout : private Std::tuple<ShapeType, StrideType> {
 // 构造函数
 __aicore__ inline Layout(Shape const& shape = {}, Stride const& stride = {})
 // 返回Shape
 template <int... I>
 __aicore__ inline Shape() {
 return get<0, I...>(static_cast<Std::tuple<Shape, Stride>&>(*this));
 }
};
```

LocalTensor/GlobalTensor

```
template <typename Engine, typename Layout>
struct LocalTensor<TensorTrait<Engine, Layout>> {
   using Iter     = typename Engine::iterator;
   using engine_type  = Engine;
   using layout_type  = Layout;
 public:
   __aicore__ constexpr LocalTensor() {}
   __aicore__ constexpr LocalTensor(Engine engine, Layout la) : rep_(Std::move(la),
Std::move(engine)) {}
   __aicore__ uint64_t GetPhyAddr() const { retu
 private:
   Std::tuple<layout_type, engine_type> rep_;
};
```

- 搬运接口
- 向量计算接口
- 矩阵计算接口

```
// 1. 定义Problem 的Shape
auto probShape = MakeShape(m, n, k);

// 2. 定义Layout
auto gmStrideA = MakeStride(k, _1); // GMA Shape(m, k)
auto gmStrideB = MakeStride(_1, n); // GMB Shape(n, k)
auto gmStrideC = MakeStride(n, _1); // GMC Shape(m, n)
// 定义L1切分的Layout, 定长 1024 * 1024 * 256
auto l1M = _128, l1N = _128, l1K = _1024;
auto gm2L1Tiler = MakeShape(l1M, l1N, k);
auto layoutL1A = MakeLayout(MakeShape(l1M, l1K));
auto layoutL1B = MakeLayout(MakeShape(l1N, l1K));
auto l0M = _128, l0N = _128, l0K = _128;
auto layoutL0A = MakeLayout(MakeShape(l0M, l0K));
…

// 基于数组申请静态内存
__cbuf__ half l1A[Cosize< layoutL1A >], l1B[Cosize< layoutL1B >];
__ca__ half l0A[Cosize< layoutL0A >];
…

// 3、构造片上Tensor
auto l1ATensor = MakeTensor(MakeL1memPtr(l1A), l1ALayout);
…

// 4、自定义数据搬运
auto gm2l1Copy = MakeCopy(CopyAtom<GM2L1Copy>{});
auto l12l0Copy = MakeCopy(CopyAtom<L12L0Copy>{});
constexpr static FixPipeTrait trait = {100};
auto l0c2OutCopy = MakeCopy(CopyAtom<L0C2OutCopy>{}, MakeTraits<trait>());

// 4、基于Layout的Tensor简化数据计算
…
Copy(gm2l1Copy, l1ATensor, globalBlkA(outMTile, outKTile));
Copy(gm2l1Copy, l1BTensor, globalBlkB(outNTile, outKTile));
Mmad(l0CTensor, l0ATensor, l0BTensor, outKTile == 0 && initMTe
…
Copy(l0c2OutCopy, globalBlkC(outMTile, outNTile), l0CTensor);
…
```

本章小结

- **下一代CUBE核心特性介绍**
  - 低比特数据类型的添加，可以进行fp4/fp8的计算；
  - 新增UB2L1&L0C2UB的通路，方便融合算子进行串联和性能优化；
  - 新增双目标模式， 融合算子可以更方便的完成CV流水并行；
- **下一代CUBE编程主要接口介绍**
  - 基础API都针对下一代的变化提供了新的接口能力；
  - 新增TensorApi的新增能力出；

CANN

目录

Part 1 当代CUBE编程基础回顾

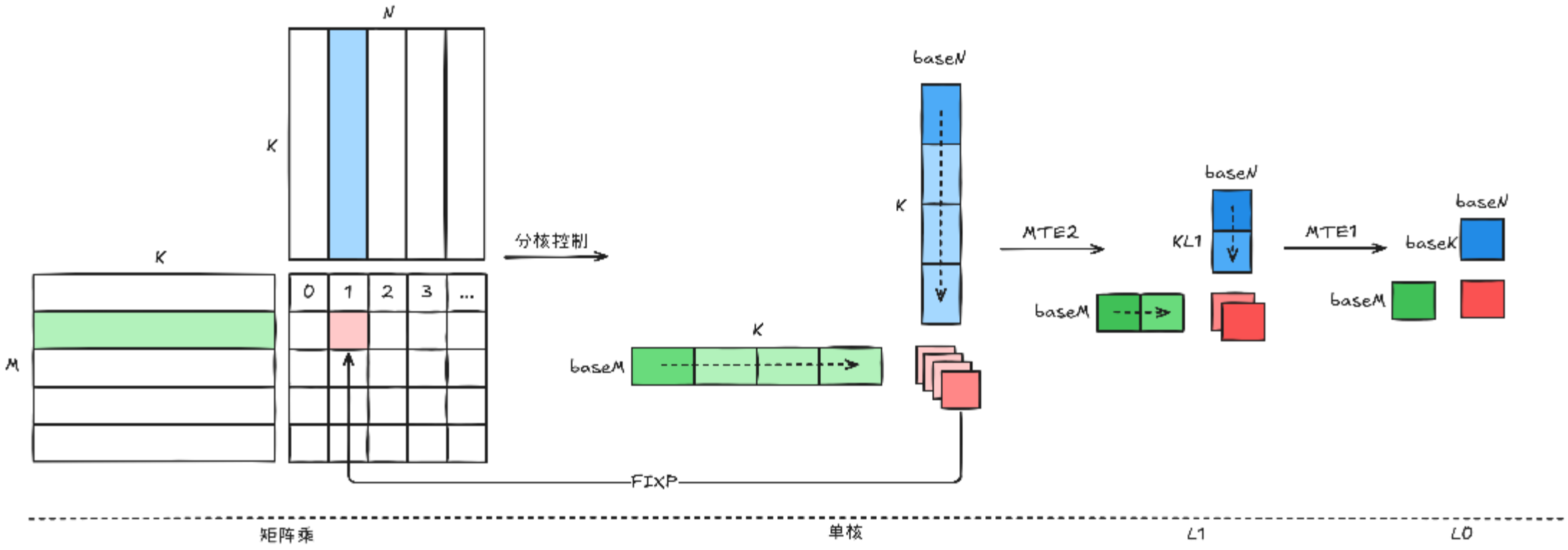
Part 2 下一代CUBE单元升级解析

Part 3 下一代CUBE编程语法及核心特性

Part 4 下一代CUBE编程实操

下一代CUBE编程实操-MatmulV3算子(1)

样例展示了如何在昇腾AI处理器的CubeCore硬件单元上使用Ascend C编程语言实现矩阵乘运算。下面是矩阵乘在新一代芯片上的执行的示意图。



下一代CUBE编程实操-MatmulV3算子(2)

```cpp
template <typename T>
__global__ __aicore__ void MatmulKernel(GM_ADDR aGm, GM_ADDR bGm, GM_ADDR cGm, ...)
{   // 单核多轮基本块计算循环
 for (uint64_t tileIdx = curBlockIdx; tileIdx < tileNum; tileIdx += blockNum) {
  AscendC::WaitFlag<AscendC::HardEvent::FIX_M>(ZERO_FLAG);
  for (uint64_t iter0 = 0; iter0 < kL1TileNum; ++iter0) {
   // ...
   AscendC::WaitFlag<AscendC::HardEvent::MTE1_MTE2>(l1BufId);
   CopyInA1<T>(aGlobal[offsetA], l1Local[offsetAL1], mL1, curKL1, k);
   CopyInB1<T>(bGlobal[offsetB], l1Local[offsetBL1], nL1, curKL1, n);
      // 设置同步MTE1等待MTE2完成搬运
   AscendC::SetFlag<AscendC::HardEvent::MTE2_MTE1>(l1BufId);
   AscendC::WaitFlag<AscendC::HardEvent::MTE2_MTE1>(l1BufId);
   for (uint64_t iter1 = 0; iter1 < kL0TileNum; ++iter1) {
         // 设置同步等待上一轮的MMAD计算
    AscendC::WaitFlag<AscendC::HardEvent::M_MTE1>(l0BufId);
    CopyInA2<T>(al0Local[l0Offset], l1Local[offsetAL1], mL1, curKL1, mL0, curKL0);
    CopyInB2<T>(bl0Local[l0Offset], l1Local[offsetBL1], nL1, curKL1, nL0, curKL0);
         // 设置同步MMAD等待MTE1完成搬运
    AscendC::SetFlag<AscendC::HardEvent::MTE1_M>(l0BufId);
    AscendC::WaitFlag<AscendC::HardEvent::MTE1_M>(l0BufId);
    // ...
    Mmad(l0cLocal, al0Local[l0Offset], bl0Local[l0Offset], mL0, nL0, curKL0, isFirstLoop);
         // 设置同步让下一轮的MTE1等待本轮MMAD完成
    AscendC::SetFlag<AscendC::HardEvent::M_MTE1>(l0BufId);
    l0PingPong++;}
        // 设置同步让下一轮的MTE2等待本轮MTE1完成
   AscendC::SetFlag<AscendC::HardEvent::MTE1_MTE2>(l1BufId);
   l1PingPong++;
  }
     // 设置同步FIXP等待MMAD完成计算
  AscendC::SetFlag<AscendC::HardEvent::M_FIX>(ZERO_FLAG);
  AscendC::WaitFlag<AscendC::HardEvent::M_FIX>(ZERO_FLAG);
     // 搬运输出到GM
  CopyOut(cGlobal[offsetC], l0cLocal, mL0, nL0, n);
     // 设置同步让下一轮的MMAD等待本轮FIXP完成
  AscendC::SetFlag<AscendC::HardEvent::FIX_M>(ZERO_FLAG);
}}
```

```cpp
template <typename T>
__aicore__ inline void CopyInA1(const AscendC::GlobalTensor<T>&
aGlobal, ...)
{
 AscendC::Nd2NzParams nd2nzParams;
 nd2nzParams.ndNum = 1;
 nd2nzParams.nValue = curML1;
 nd2nzParams.dValue = curKL1;
 // ...
 AscendC::DataCopy(al1Local, aGlobal, nd2nzParams);
}
```

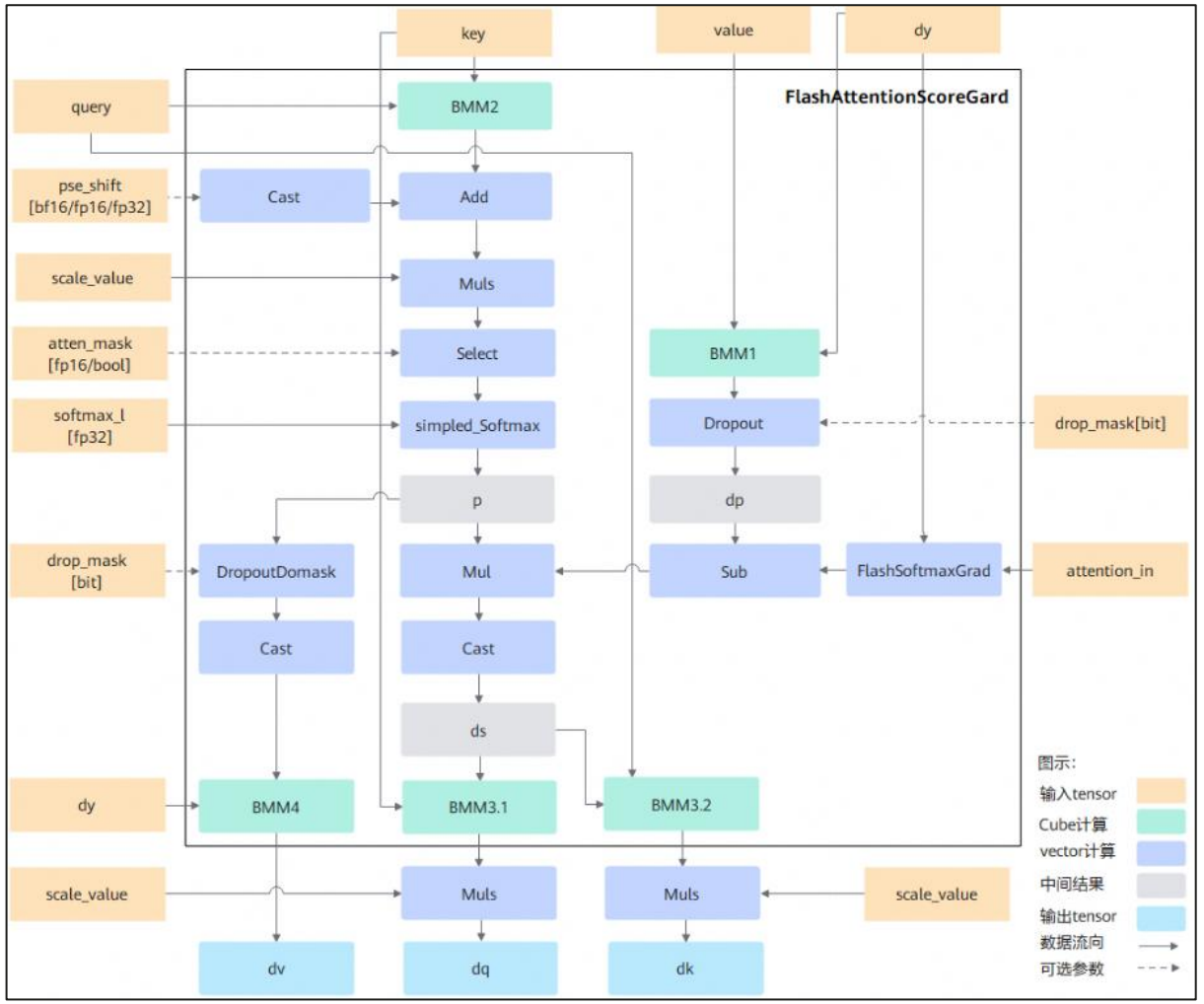
```cpp
template <typename T>
__aicore__ inline void CopyInA2(const AscendC::LocalTensor<T>&
al0Local, /*...*/)
{
 AscendC::LoadData2DParamsV2 loadDataParams;
 loadDataParams.mStartPosition = 0;
 loadDataParams.kStartPosition = 0;
 loadDataParams.mStep = CeilDiv(mL0, AscendC::BLOCK_CUBE);
 loadDataParams.kStep = CeilDiv(kL0, AscendC::BLOCK_CUBE);
 // ...
 AscendC::LoadData<T>(al0Local, al1Local, loadDataParams);
}
```

```cpp
template <typename T>
__aicore__ inline void CopyOut(const AscendC::GlobalTensor<T>&
cGlobal, /*...*/)
{
 AscendC::DataCopyCO12DstParams intriParams;
 intriParams.nSize = baseN;
 intriParams.mSize = baseM;
 intriParams.dstStride = n;
  // ...
 AscendC::SetFixpipeNz2ndFlag(1, 1, 1);
 AscendC::DataCopy(cGlobal, c1Local, intriParams);
}
```

示例来源：https://gitcode.com/cann/ops-samples/tree/master/Samples/0_Introduction/matmul

下一代CUBE在算子中的应用示范-FAG算子(1)

新架构延续AIC/AIV分离设计，保留并行执行优势，新增L0C->UB->L1高速数据通路，显著降低CV交互成本，提升流水并行效率。相比Atlas A2复杂的tiling策略，新架构仅需单一基本块即可实现更优性能，大幅简化开发复杂度。



FAG计算流程图

```
// 算子实现伪码
// 1. 进行q@k && value@dy的矩阵计算, 基本块为128*128, 结果存储在L0C上
Matmul((S1_i,D)x(D,S2_i));
// 2. 通过L0C2UB 双目标输出L0C到UB上面, 将128*128均分到两个V核的UB上
FixPipe(S1_i / 2,S2_i);
// 3. CV 核间同步(V0&V1等待CUBE执行完毕)
// 4. Vector侧计算，执行DropOut & SimpleSoftmax等计算
VectorCompute(S1_i / 2,S2_i);
// 5. Vector计算结束, 得到输出数据，通过DataCopy的UB2L1的能力，将两个V核的数据聚合拷贝到L1上
DataCopy(S1_i/2*S2_i);
// 6. CV核间同步(CUBE等V0 & V1搬运到L1后进行后续计算)：DQ & DKV计算
Matmul((S1_i,S2_i)*(S2_i,D));
Matmul((S2_i,S1_i)*(S1_i,D));
```

CANN

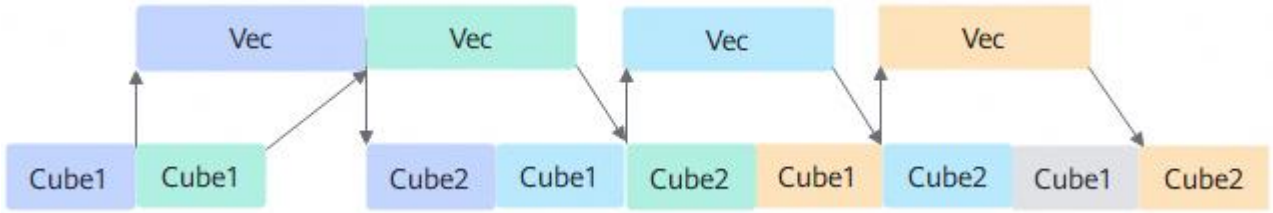
示例来源：ops-transformer/attention/flash_attention_score_grad · CANN/ops-transformer - AtomGit | GitCode

下一代CUBE在算子中的应用示范-FAG算子

融合算子通常包含了Vector计算和Cube计算，对于FA算子，V侧的计算是依赖C侧的计算结果的，如果只关注V侧流水，不关注C侧，则C侧与V侧很有可能是串行流水的效果，不能达到并行计算的目的，无法使得融合算子性能达到最优，从而有了CV流水设计。此外，CV流水在不同算子情况下，表现的现象也是不一致的，FA/FAG的Cube双发机制，又称为CV间preload流水，可实现两种场景下的流水优化：

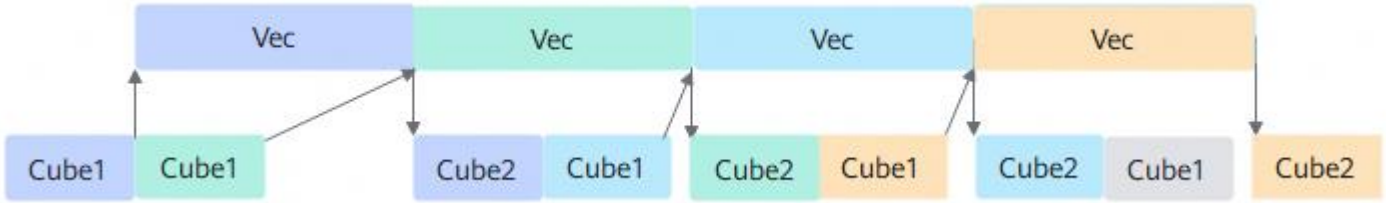
- C侧总耗时 > V侧总耗时

该场景流水特征下，Vector计算节点少，计算速度快，在Cube双发机制下，提前发射两块Cube计算，Cube1、Cube2计算可以衔接，使得Cube利用率最高，达成Cube bound。



- C侧总耗时 < V侧总耗时

该场景流水特征下，Vector计算节点多，Vector计算是瓶颈，C侧的搬运不足以覆盖V侧的流水，因此需要进行CV流水排布，尽量达到CV并行的效果，最通用的优化手段是C侧提前发射。



C侧连续发射两块Cube计算，这样可以保证V侧计算完上一轮时，可以立马启动当前轮的计算，而不用等待Cube1的数据。这样可以使V侧一直在工作，达成Vector bound。

示例来源：ops-transformer/attention/flash_attention_score_grad · CANN/ops-transformer - AtomGit | GitCode

下一代CUBE在算子中的应用示范-FAG算子(3)

```
// 整体代码逻辑
while (true) {
  // 1. Vector计算softmaxGrad, 该内存不依赖C侧输入，因此不需要等待C侧结果
  this->vecBlock.ProcessVec1(this->constInfo, prevRunInfo);
  CrossCoreWaitFlag<SYNC_MODE, PIPE_V>(SYNC_C1_TO_V2_FLAG[(taskId + 1) & 1]);
  CrossCoreWaitFlag<SYNC_MODE, PIPE_V>(SYNC_C2_TO_V2_FLAG[(taskId + 1) & 1]);
  // 2. 进行DyV & QK的矩阵计算, 计算结果搬出到UB中, 需要Set同步信号量
  this->cubeBlock.IterateMmDyV(mm1ResTensor, this->constInfo, …);
  if ASCEND_IS_AIC {
    CrossCoreSetFlag<SYNC_MODE, PIPE_FIX>(SYNC_C1_TO_V2_FLAG[taskId & 1]);
  }
  this->cubeBlock.IterateMmQK(mm2ResTensor, this->constInfo, …);
  if ASCEND_IS_AIC {
    CrossCoreSetFlag<SYNC_MODE, PIPE_FIX>(SYNC_C2_TO_V2_FLAG[taskId & 1]);
  }
  // 3. 进行后续的Vector计算, 依赖DyV&QK的CUBE计算结果，前序已经插入核间同步
  this->vecBlock.ProcessVec3(dSL1Buffer, mm1ResTensor, …);
  this->vecBlock.ProcessVec4(pL1Buffer, mm2ResTensor, this->constInfo, prevRunInfo);
  if ASCEND_IS_AIV {
    CrossCoreSetFlag<SYNC_MODE, PIPE_MTE3>(SYNC_V3_TO_C3_FLAG);
    CrossCoreSetFlag<SYNC_MODE, PIPE_MTE3>(SYNC_V4_TO_C5_FLAG);
  }
  // 4. 进行Q、K、V的结果运算，依赖Vector数据计算完成
  if ASCEND_IS_AIC {
    CrossCoreWaitFlag<SYNC_MODE, PIPE_MTE1>(SYNC_V3_TO_C3_FLAG);
    CrossCoreWaitFlag<SYNC_MODE, PIPE_MTE1>(SYNC_V4_TO_C5_FLAG);
  }
  this->cubeBlock.template IterateMmDsK<CALC_TYPE, BaseClass::IS_DQ_WRITE_UB>(…);
  this->cubeBlock.template IterateMmDsQ<CALC_TYPE, BaseClass::IS_DK_WRITE_UB>(…);
  this->cubeBlock.template IterateMmPDy<CALC_TYPE, BaseClass::IS_DV_WRITE_UB>(…);
}
```

```
// CUBE核心代码
TEMPLATES_DEF_NO_DEFAULT
__aicore__ inline void
FAGBlockCube<TEMPLATE_ARGS>::IterateMmDyV(LocalTensor<CALC_TYPE>
&mm1ResTensor, FagConstInfo &constInfo, FagRunInfo &runInfo,
PreloadArgs<IS_ROPE> &preloadArgs)
{
  // ...
  for (uint32_t n = 0; n < nLoops; ++n) {
    // 搬入
    if (isCopyRight) {
      DataCopy(vL1Tensor, this->valueGm[runInfo.commonRunInfo.valueOffset +
gmNOffset], nd2NzParams);
    }
    // 计算
    MMParam param = {
      (uint32_t)runInfo.commonRunInfo.s1RealSize,
      ...};
    // 搬出
    FixpipeParamsC310<CO2Layout::ROW_MAJOR> fixpipeParams;
    fixpipeParams.dualDstCtl = 1;
    Fixpipe<CALC_TYPE, CALC_TYPE,
      PFA_CFG_ROW_MAJOR_UB>(mm1ResTensor[ubOffset],
      mm1L0CBuffer.GetTensor<CALC_TYPE>(), fixpipeParams);
  }
}
```

示例来源：ops-transformer/attention/flash_attention_score_grad · CANN/ops-transformer - AtomGit | GitCode

本章小结

- **本章讲述了CUBE编程中，新的特性在实际算子中的应用**
  - FAG算子利用CV新增通路特性， 可以写出比当代更简洁的切分策略和循环策略;
  - MatmulV3算子则展示了新一代架构下的典型Matmul的基础场景，后续样例仓中会补充FP4, FP8输入数据类型;

CANN

附录

Ascend API样例仓: https://gitcode.com/cann/asc-devkit/tree/master/examples

算子样例仓: https://gitcode.com/cann/ops-samples/tree/master/Samples

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN